🟥 Agents in LangChain

---

1. What is an Agent?

- **Definition**:
  An **Agent** is an LLM-powered system that can **autonomously decide** which tools to use, in what order, and how, to achieve a user's goal.
- **Difference from Tool Calling**:
  - Tool Calling → LLM suggests tools, **programmer executes**.
  - Agent → LLM itself **decides, executes, and orchestrates** multiple tools until goal is achieved.

👉 Think of Agents as:

- Tools = *hands & legs*
- LLM = *brain*
- Agent = *full human (brain + body working together automatically)*

---

2. Why Agents?

- Without agents → we had to manually:
  - Bind tool
  - Wait for tool call
  - Run it ourselves
  - Inject result back
- With agents → the **loop is automated**.
- Useful for:
  - Searching web + summarizing
  - Fetching APIs + formatting answers
  - Multi-step workflows

---

3. Agent Workflow

1. **User query** → "Convert 10 USD to INR and tell me about INR in Wikipedia."
2. **Agent reasoning** →
   - Step 1: Use Currency API tool
   - Step 2: Use Convert tool
   - Step 3: Use Wikipedia tool
   - Step 4: Combine results
3. **Agent executes automatically** until goal achieved.

---

4. Code Walkthrough

🔹 **Step 1: Import & Setup**

```
from langchain_openai import ChatOpenAI
from langchain.agents import initialize_agent, AgentType, load_tools
from langchain_core.tools import tool

llm = ChatOpenAI(model="gpt-4o-mini")
```

🔹 **Step 2: Define Custom Tools**

```
@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
```

```
    return a * b

@tool
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b
```

🔹 **Step 3: Load Built-in Tools**

```
tools = load_tools(["ddg-search"])  # DuckDuckGo search
tools += [multiply, add]
```

🔹 **Step 4: Initialize Agent**

```
agent = initialize_agent(
    tools=tools,
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,  # reasoning + acting
    verbose=True
)
```

🔹 **Step 5: Run Agent**

```
agent.run("Search the current USD to INR conversion rate and multiply it with 10.")
```

👉 Behind the scenes:

- LLM → plans steps
- Calls API tool → gets conversion
- Calls multiply tool → gets result
- Returns final answer

---

5. Agent Types in LangChain

- **ZERO_SHOT_REACT_DESCRIPTION**
  - Default type.
  - Uses prompt reasoning: *"Thought → Action → Observation → Answer"*.
- **STRUCTURED_CHAT_ZERO_SHOT**
  - Handles structured tool input/output.
- **OPENAI_FUNCTIONS**
  - Uses OpenAI's function calling mechanism.

👉 In practice, **ZeroShotReactDescription** is most common.

---

6. Key Interview Takeaways

- **Agent vs Tool Calling:**
  - Tool Calling = semi-automatic, programmer still executes.
  - Agent = fully automatic orchestration.
- **How does an Agent decide?**
  - LLM prompt includes available tools + descriptions.
  - LLM "thinks" step by step, calls tools in sequence.
- **Risks with Agents:**
  - **Hallucination** → calling irrelevant tools.
  - **Loops** → infinite calling if not careful.
  - **API cost** → many tool calls = expensive.

- **Best Practices:**
  - Restrict tools to only what's needed.
  - Use max_iterations to prevent infinite loops.
  - Monitor using **LangSmith** or logs.

---

7. Real-World Applications

- **Travel Bot** – queries flights, hotels, weather APIs in sequence.
- **Finance Assistant** – fetches stock data, applies formulas, returns insights.
- **Research Copilot** – web search, Wikipedia, summarize, cross-compare.

✅ **Summary:**

- Agents = autonomous LLM + tools.
- They can plan, call, and execute without programmer intervention.
- In LangChain: initialize_agent() with tools + LLM.
- AgentTypes → ZeroShotReactDescription (default).
- Key for building **agentic AI apps** like Jarvis-style assistants.